Relational Databases / Dependencies

Nicolae-Cosmin Vârlan

Basics of relational model

- Let U be a set of attributes: $U = \{A_1, A_2, \ldots, A_n\}$;
- $dom(A_i)$ - is the domain of the atribute A_i;

We define a *tuple* over U as being the function:

$$\varphi : U \rightarrow \bigcup_{1 \leq i \leq n} dom(A_i) \quad \text{such that } \varphi(A_i) \in dom(A_i), 1 \leq i \leq n$$

Consider the values v_i such that $v_i = \varphi(A_i)$.

We denote by $\{A_1 : v_1,\ A_2 : v_2,\ \ldots,\ A_n : v_n\}$ the association between the attributes in U and their values.

We can consider the ordered set: $(A_1, A_2, \ldots, A_n)$; in this case, we can omit the atributes and only enumerate the values (in the same order): $(v_1, v_2, \ldots, v_n)$.

Basics of relational model

Consider the ordered set $(A_1, A_2, \ldots A_n)$. For each tuple φ, there exists a vector $(v_1, v_2, \ldots v_n)$ of values s.t. $\varphi(A_i) = v_i,\ 1 \leq i \leq n$.

For a vector $(v_1, v_2, \ldots v_n)$ with $v_i \in dom(A_i),\ 1 \leq i \leq n$ there exists a tuple φ s.t. $\varphi(A_i) = v_i$.

In practice the set of attributes is always considered as beeing ordered (the columns in a table are in a specific order).

Basics of relational model

A set of tuples over U is caled a *relation* and is denoted by r.

r can vary in time (for example by adding or removing elements) but not in its structure (tuples will always be over the same set U of attributes).

Example:
$r = \{(v_{11}, v_{12}, \ldots v_{1n}), (v_{21}, v_{22}, \ldots v_{2n}), \ldots, (v_{m1}, v_{m2}, \ldots v_{mn})\}$.

The structure of the relation will be denoted by $R[U]$ where R is the *relation name* and U is the set of *attributes*.

Equivalent notations: $R(U)$, $R(A_1, A_2, \ldots, A_n)$, $R[A_1, A_2, \ldots, A_n]$.

$R[U]$ is also called *relation schema*.

By r built over $R[U]$ we denote the relation r corresponding to $R[U]$.

Ex: $R[U]$ might be: students(id int, name char(10), scholarship int)

Basics of relational model

In practice, a relation r can be represented as a matrix:

$$r: \begin{array}{cccc} A_1 & A_2 & \dots & A_n \\ \hline v_{11} & v_{12} & \dots & v_{1n} \\ \dots & \dots & \dots & \dots \\ v_{m1} & v_{m2} & \dots & v_{mn} \\ \hline \end{array}$$

where $(v_{i1}, v_{i2}, \dots, v_{in})$ is a tuple from r, $1 \leq i \leq m$ and $v_{ij} \in dom(A_j)$, $1 \leq j \leq n, 1 \leq i \leq m$

We will denote by t_i the line i from the matrix:
$t_i = (v_{i1}, v_{i2}, \dots, v_{in})$

Basics of relational model

A finite set D of relation schemas is called a *database schema*. Formally, $D = \{R_1[U_1], \ldots, R_h[U_h]\}$ where $R_i[U_i]$ is a relation schema, $1 \leq i \leq h$.

A *database over D* is a bijection that associates a relation r to a relation schema in D.

Example:
$r_1, r_2, \ldots r_h$ is a database over $D = \{R_1[U_1], \ldots, R_h[U_h]\}$.

Considering that D is an ordered set $D = (R_1[U_1], \ldots, R_h[U_h])$, we can denote a database by $(r_1, r_2, \ldots r_h)$.

Operations in Relational Model

There are two types of operands that can be used to manipulate a set of relations:

- The ones specific to *set theory*: Union($\cup$), Intersection ($\cap$), Diference($-$), Cartesian Product($\times$)
- Relational algebras specific operations: Projection (π), Selection(σ), Renaming(ρ), Natural Join($\bowtie$), θ-Join, equijoin, Semijoin($\ltimes$ and $\rtimes$), Antijoin($\triangleright$), Division($\div$), Left Outer Join ($⟕$), Right Outer Join ($⟖$), Full Outer Join($⟗$)

Sets Operations in Relational Model - *Union*

Union, Intersection and Difference are done between two sets of relations having EXACTLY the same relation schemas.
The Cartesian Product can be done only between two realtions having different schemas (even more, if r_1 is over $R_1[U_1]$ and r_2 is over $R_2[U_2]$ then $U_1 \cap U_2 = \emptyset$).

Union of two relations r_1 and r_2, both over the same set of attributes U, is a new relation denoted by $r_1 \cup r_2$ and defined as:

$$r_1 \cup r_2 = \{t \mid t = \text{tuple}, \ \ t \in r_1 \ \vee \ t \in r_2\}$$

In SQL, this is realized by using the word UNION between two different select, both having EXACTLY the same schema:

```
SELECT * FROM students WHERE year=1
  UNION
  SELECT * FROM students WHERE year=3;
```

Sets Operations in Relational Model - *Difference*

The Difference of two relations r_1 and r_2, both over the same set of attributes U, is a new relation denoted by $r_1 - r_2$ defined as:

$$r_1 - r_2 = \{t \mid t = tuple, \;\; t \in r_1 \;\wedge\; t \notin r_2\}$$

In SQL, the keyword for achieving this is MINUS.
For example, you can get all the students in the second year that don't have a scholarship by getting all the students in the second and eliminating the ones that have a scholarship:

```
SELECT * FROM students WHERE year=2
  MINUS
  SELECT * FROM students WHERE scholarship IS NOT NULL;
```

Sets Operations in Relational Model - *Intersection*

The Intersection of two relations r_1 and r_2, both over the same set of attributes U, is a new relation denoted by $r_1 \cap r_2$ defined as:

$$r_1 \cap r_2 = \{t \mid t = tuple, \;\; t \in r_1 \;\wedge\; t \in r_2\}$$

In SQL, the keyword used is INTERSECT.

Example:

```
SELECT * FROM students WHERE year=2
  INTERSECT
  SELECT * FROM students WHERE scholarship IS NOT NULL;
```

You can also easily see that: $r_1 \cap r_2 = r_1 - (r_1 - r_2)$

Sets Operations in Relational Model - *Cartesian Product*

The Cartesian Product of two relations r_1 defined over $R_1[U_1]$ and r_2 defined over $R_2[U_2]$ with $U_1 \cap U_2 = \emptyset$ is a new relation denoted by $r_1 \times r_2$ defined as it follows:

$$r_1 \times r_2 = \{t \mid t = tuple\ over\ U_1 \cup U_2,\ \pi_{U_1}[t] \in r_1\ \wedge\ \pi_{U_2}[t] \in r_2\}$$

$\pi_{U_1}[t]$ stands for the set of values in t restricted to the set of attributes in U_1.

For example, if $t = (v_1, v_2, v_3, v_4)$ and $U_1 = (A_1, A_2)$ then $\pi_{U_1}[t] = (v_1, v_2)$.

Keep in mind that $U_1 \cap U_2 = \emptyset$.

Sets Operations in Relational Model - *Cartesian Product*

If, however, an attribute A_i is similar to both U_1 and U_2, it will be identified as $U_1.A_i$ and $U_2.A_i$ respectively. The values will be treated as if they would have nothing in common.

In SQL, the Cartesian Product of two tables is obtained by selecting both tables (separated with a comma):

```
SELECT * FROM students, profs;
```
(ORACLE)

```
SELECT * FROM students CROSS JOIN profs;
```
(SQL standard)

Relational Algebra Operations

On sets, the elements were tuples of values, each value corresponding to a specific attribute.

Some operations have the restriction that the tuples from both relations are buit over the same set of attributes. We can manipulate those sets of attributes (and the corresponding values) by the means of specific operators defined with this purpose in mind. These operators are called Relational Algebra Operators.

For example, we can't use union over the tables Studenti and Profesori because their structure is different. However we should be able to make a union over the first or the second name (or both) columns.

Relational Algebra Operations - *Projection:* π

Consider:

- $R[U]$ = a relation schema;
- $X \subseteq U$;
- t = tuple over $R[U]$ ($t \in r$).

It is called the *projection of t with respect to (w.r.t.) X* and denoted by $\pi_X[t]$, the restriction of t to the set of attributes X.

Example:
If $U = (A_1, A_2, \ldots, A_n)$ then $t = (v_1, v_2, \ldots, v_n)$.

Let $X = (A_{i_1}, A_{i_2}, \ldots, A_{i_k})$, $1 \leq i_1 < i_2 < \ldots < i_k \leq n$.
then, we have that $\pi_X[t] = (v_{i_1}, v_{i_2}, \ldots, v_{i_k})$;

Relational Algebra Operations - *Projection:* π

If r is a relation over $R[U]$ and $X \subseteq U$, then *the projection of r w.r.t. X* is $\pi_X[r] = \{\pi_X[t] \mid t \in r\}$

Example:
Let $U = (A_1, A_2, \ldots, A_n)$. A relation r over U is a set of tuples:
$r = \{(v_{11}, v_{12}, \ldots v_{1n}), (v_{21}, v_{22}, \ldots v_{2n}), \ldots, (v_{m1}, v_{m2}, \ldots v_{mn})\}$.

Considering $X = (A_{i_1}, A_{i_2}, \ldots, A_{i_k})$, $1 \leq i_1 < i_2 < \ldots < i_k \leq n$.
$\pi_X[r] = \{(v_{1_{i_1}}, v_{1_{i_2}}, \ldots v_{1_{i_k}}), (v_{2_{i_1}}, \ldots v_{2_{i_k}}), \ldots, (v_{m_{i_1}}, \ldots v_{m_{i_k}})\}$

In SQL, the projection is achieved by selecting certain columns from the table. For example:

```
SELECT fname, lname FROM students;
```

Relational Algebra Operations - *Projection:* π

As an example, we will select all the persons (professors and students) that are forming the faculty:

```
SELECT lname, fname FROM students
  UNION
  SELECT lname, fname FROM profs;
```

Relational Algebra Operations - *Selection:* σ

Let r be a relation over $R[U]$.
We define the *elementary selection expressions* as being: $A_1 \theta A_2$, $A_1 \theta c$ and $c\, \theta A_2$, where $A_1, A_2 \in U$, c is a constant value and θ is a comparision operator (e.g. equality, less than, etc.)

If φ_1 and φ_2 are selection expressions (elementary or not), then the following are selection expressions: (φ_1), $\neg\varphi_1$, $\varphi_1 \wedge \varphi_2$, $\varphi_1 \vee \varphi_2$.

Relational Algebra Operations - *Selection:* σ

Let φ be a selection expression:

- Considering that $\varphi = A\theta B$ then t satisfy φ iff $\pi_A[t]\ \theta\ \pi_B[t]$,
- Considering that $\varphi = A\theta c$ then t satisfy φ iff $\pi_A[t]\ \theta\ c$,
- Considering that $\varphi = c\theta B$ then t satisfy φ iff $c; \theta\ \pi_B[t]$,
- when $\varphi = \neg\varphi_1$ then t satisfy φ if t does not satisfy φ_1,
- when $\varphi = \varphi_1 \wedge \varphi_2$ then t satisfy φ if t satisfy both φ_1 and φ_2,
- when $\varphi = \varphi_1 \vee \varphi_2$ then t satisfy φ if t satisfy either φ_1 or φ_2.

If φ is a selection expression, then the selection φ over a relation r, denoted by $\sigma_\varphi(r)$, is defined as:
$\sigma_\varphi(r) = \{t | t = tuple\ over\ R[U], t\ satisfy\ \varphi\}$

Relational Algebra Operations - *Selection:* σ

In SQL, selection is obtained by using a set of clauses that selects only certain rows from the table.

For example:
SELECT * FROM students WHERE year=2 and scholarship IS NULL;

In this example, φ_1 is $AN = 2$, φ_2 is $bursa\ IS\ NULL$, $\varphi = \varphi_1 \wedge \varphi_2$ and r is the set of rows in students table.

The result is the selection of the students in second year that don't have a scholarship.

Relational Algebra Operations - *Renaming:* ρ

The purpose of the *Renaming* operator is to change an attribute A_1 into an attribute A_1'. It will not change the domain value of the attribute A_1 but only its name.

The renaming operation is denoted by $\rho_{A_1/A_1'}(r)$ and only changes the name of the attribute.

In SQL, Renaming is achieved by using the AS keyword.

Example:
SELECT LNAME AS "Name" FROM students;

Relational Algebra Operations - *Natural Join:* $\bowtie$

Let:

- r_1 be a relation over $R_1[U_1]$;
- r_2 be a relation over $R_2[U_2]$;

It is called *Natural Join* of relations r_1 and r_2, the relation $r_1 \bowtie r_2$ over $U_1 \cup U_2$ defined by:

$$r_1 \bowtie r_2 = \{t \mid t \text{ tuple over } U_1 \cup U_2,\ t[U_i] \in r_i,\ i = 1, 2\}$$

If R is the name for the relation over $U_1 \cup U_2$ then $r_1 \bowtie r_2$ is defined over $R[U_1 \cup U_2]$

For simplicity, we will denote $U_1 \cup U_2$ by U_1U_2.

Relational Algebra Operations - *Natural Join:* $\bowtie$

Example:

Let $R_1[A, B, C, D]$, and $R_2[C, D, E]$ and r_1, r_2 s.t.:

r_1 :

A	B	C	D
0	1	0	0
1	1	0	0
0	0	1	0
1	1	0	1
0	1	0	1

r_2 :

C	D	E
1	1	0
1	1	1
0	0	0
1	0	0
1	0	1

Then: $r_1 \bowtie r_2$:

A	B	C	D	E
0	1	0	0	0
1	1	0	0	0
0	0	1	0	0
0	0	1	0	1

Relational Algebra Operations - *Natural Join:* $\bowtie$

In SQL, the natural join can be achieved by multiple means. Let's see some querries in which each students is associated with the marks he took:

```
SELECT lname, value FROM students NATURAL JOIN grades;
```

Some properties of Natural Join:

- $r_1 \bowtie r_2[U_1] \subseteq r_1$
- $r_2 \bowtie r_1[U_2] \subseteq r_2$

If $X = U_1 \cap U_2$ and:
$r_1' = \{t_1 | t_1 \in r_1, \exists t_2 \in r_2$ s.t. $t_1[X] = t_2[X]\}$ and $r_1" = r_1 - r_1'$,
$r_2' = \{t_2 | t_2 \in r_2, \exists t_1 \in r_1$ s.t. $t_1[X] = t_2[X]\}$ and $r_2" = r_2 - r_2'$,
then: $r_1 \bowtie r_2 = r_1' \bowtie r_2'$, $r_1 \bowtie r_2[U_1] = r_1'$, $r_2 \bowtie r_1[U_2] = r_2'$.

If $\overline{r_1} \subseteq r_1, \overline{r_2} \subseteq r_2$ and $\overline{r_1} \bowtie \overline{r_2} = r_1 \bowtie r_2$ then $r_1' \subseteq \overline{r_1}$ and $r_2' \subseteq \overline{r_2}$

If $U_1 \cap U_2 = \emptyset$ then $r_1 \bowtie r_2 = r_1 \times r_2$.

The Natural Join can be extended over multiple relations:

Consider r_i a relation over $R_i[U_i], \forall i = \overline{1,h}$ then:

$$r_1 \bowtie r_2 \bowtie \ldots \bowtie r_h = \{t | t \text{ tuple over } U_1, \ldots U_h, \text{ s.t. } t[U_i] \in r_i, i = \overline{1,h}\}$$

Equivalent notations:

- $r_1 \bowtie r_2 \bowtie \ldots \bowtie r_h$
- $\bowtie \langle r_i, i = \overline{1,h} \rangle$
- $* \langle r_i, i = \overline{1,h} \rangle$

The Natural Join operation is associative.

Relational Algebra Operations - *Theta Join (θ-join)*

Let r_1 be a relation over $R_1[U_1]$, r_2 a relation over $R_2[U_2]$ with $A_{\alpha_1}, A_{\alpha_2}, \ldots A_{\alpha_k} \in U_1$ and $B_{\beta_1}, B_{\beta_2}, \ldots B_{\beta_k} \in U_2$ and $\theta_i : dom(A_{\alpha_i}) \times dom(B_{\beta_i}) \rightarrow \{true, false\}, \forall i = \overline{1,k}$.

The θ-join of two relations r_1 and r_2, denoted by $r_1 \bowtie_\theta r_2$, is defined as:

$$r_1 \bowtie_\theta r_2 = \{(t_1, t_2) | t_1 \in r_1, t_2 \in r_2, \pi_{A_{\alpha_i}}[t_1] \theta_i \pi_{B_{\beta_i}}[t_2], i = \overline{1,k}\}$$

where $\theta = (A_{\alpha_1} \theta_1 B_{\beta_1}) \wedge (A_{\alpha_2} \theta_2 B_{\beta_2}) \wedge \ldots \wedge (A_{\alpha_k} \theta_k B_{\beta_k})$

If θ_i is the equality operator then the θ-join is also called an equijoin.

Relational Algebra Operations - *Theta Join (θ-join)*

Observation 1: a θ-join that is always $true$ for any θ_i is a Cartesian Product: $r_1 \bowtie_{true} r_2 = r_1 \times r_2$.

Observation 2: a θ-join can be considered as beeing a special (not based on equality) filtering (selection) of a Cartesian Product: $r_1 \bowtie_{\theta} r_2 = \sigma_{\theta}(r_1 \times r_2)$

A SQL example:

```
SELECT s.fname, p.fname FROM students s, profs p
  WHERE s.fname > p.fname;
```

Relational Algebra Operations - *Semijoin:* $\ltimes$ *and* $\rtimes$

A left semijoin only selects rows from the left relation ($\ltimes$) that have a corespondent (as in natural join) in the right relation.

Formally, we define the left semijoin of two relations r_1 over $R_1[U_1]$ and r_2 over $R_2[U_2]$ as being:

$$r_1 \ltimes r_2 = \pi_{U_1}(r_1 \bowtie r_2)$$

The right semijoin is defined the same way (by taking the rows from r_2 that have a corespondent in r_1).

Relational Algebra Operations - *Antijoin:* $\triangleright$

The remaining rows (that are not taken) after a left semijoin, is the resulting set of an Antijoin operation.

Formally, we define the Antijoin of two relations r_1 over $R_1[U_1]$ and r_2 over $R_2[U_2]$ as being:

$$r_1 \triangleright r_2 = r_1 - \pi_{U_1}(r_1 \bowtie r_2)$$

Relational Algebra Operations - *Left Outer Join:* $⟕$

Let r_1 and r_2 be two relations such that not all the tuples in r_1 have a corespondent in r_2.
A Left Outer Join operations of the two relations r_1 and r_2 is a union between the $r_1 \bowtie r_2$ and the tuples from r_1 that are not used in the natural join and which are completed with the NULL values for the attributes in U_2.

$$r_1 ⟕ r_2 = r_1 \bowtie r_2 \text{ UNION } \pi_{U_1 U_2}(r_1 - (r_1 \bowtie r_2))$$

Right Outer Join is defined the same way but takes the tuples in r_2 that don't have a matching tuple in r_1 and complete them with nulls for the attributes in U_1 and unions with the natural join of the two relations.

Relational Algebra Operations - *Full Outer Join:* ⟗

Takes all the tuples from both relations, matching them in the result or completing them with NULL if they don't have a corespondent.

Relational Algebra Operations - *Full Outer Join:* ⟗

Some examples (care with the equality):

```
SELECT * FROM studenti LEFT JOIN profesori ON
  studenti.prenume = profesori.prenume;
```

(All students and associations with the teachers having the same first name)

```
SELECT * FROM studenti RIGHT JOIN profesori ON
  studenti.prenume = profesori.prenume;
```

(some students that are associated with some teachers and also the rest of the teachers)

```
SELECT * FROM studenti FULL JOIN profesori ON
  studenti.prenume = profesori.prenume;
```

(all students and all teachers and asociation when it's possible)

Functional Dependencies

Functional Dependencies

Let $X, Y \subseteq U$. We will denote a functional dependency by $X \to Y$.

A relation r over $R[U]$ satisfy the *functional dependency* $X \to Y$ iff:

$$(\forall t_1, t_2)(t_1, t_2 \in r)[t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]]$$

If $X = \emptyset$ then $\emptyset \to Y$ iff $(\forall t_1, t_2)(t_1, t_2 \in r)[t_1[Y] = t_2[Y]]$

If $Y = \emptyset$ then $\forall r$ over $R[U]$ we have that $X \to \emptyset$

If r satisfy $X \to Y$, then there exists a function $\varphi : r[X] \to r[Y]$ defined by $\varphi(t) = t'[Y]$, where $t' \in r$ and $t'[X] = t \in r[X]$.
If r satisfy $X \to Y$ we will say that Y functionally depends on X in r.

Properites of Functional Dependencies

FD1. (Reflexivity) If $Y \subseteq X$, then r satisfy $X \to Y$, $\forall r \in U$.

FD2. (Extension) If r satisfy $X \to Y$ and $Z \subseteq W$, then r satisfy $XW \to YZ$.

FD3. (Transitivity) If r satisfy $X \to Y$ and $Y \to Z$, then r also satisfy $X \to Z$.

FD4. (Pseudotransitivity) If r satisfy $X \to Y$ and $YW \to Z$, then r satisfy $XW \to Z$.

Properites of Functional Dependencies

FD5. (Union) If r satisfy $X \to Y$ and $X \to Z$, then r satisfy $X \to YZ$.

FD6. (Decomposition) If r satisfy $X \to YZ$, then r satisfy $X \to Y$ and $X \to Z$.

FD7. (Proiectability) If r is over $R[U]$ and satisfy $X \to Y$ and $X \subset Z \subseteq U$, then $r[Z]$ satisfy $X \to Y \cap Z$

FD8. (Inverse Proiectability) If $X \to Y$ is satisfied by a projection of r, then $X \to Y$ is satisfied by r.

Functional Dependencies - consequence and coverage

If Σ is a set of functional dependencies over $R[U]$ then we say that $X \to Y$ *is a consequence from* Σ if any other relation that satisfy all the dependencies in Σ also satisfy $X \to Y$.
Notation: $\Sigma \models X \to Y$

Let $\Sigma^* = \{X \to Y | \Sigma \models X \to Y\}$. Let $\Sigma_1 =$ a set of functional dependiences. Σ_1 is a *coverage* for Σ^* if $\Sigma_1^* = \Sigma^*$.

Properites of Functional Dependencies

Proposition

For each set Σ of functional dependencies there exist a coverage Σ_1 for Σ^*, such that all the dependencies in Σ_1 are of the form $X \to A$, A being an atribute from U.

Proposition

$\Sigma \models X \to Y$ *iff* $\Sigma \models X \to B_j$ *for* $j = \overline{1, h}$, *where* $Y = B_1 \ldots B_h$.

Deduction rules

Let $\mathcal{R}$ be a set of deduction rules for functional dependecies and Σ a set of functional dependencies. We say that $X \to Y$ is a *proof* in Σ using the rules $\mathcal{R}$ and will denote it by $\Sigma \vdash_{\mathcal{R}} X \to Y$, if there exists a sequence of steps $\sigma_1, \sigma_2, \ldots, \sigma_n$, such that:

- $\sigma_n = X \to Y$ and
- $\forall i = \overline{1, n}$, $\sigma_i \in \Sigma$ or, there exists a rule in $\mathcal{R}$ of the form $\frac{\sigma_{j_1}, \sigma_{j_2}, \ldots \sigma_{j_k}}{\sigma_i}$, where $j_1, j_2, \ldots, j_k < i$.

Deduction rules

According to properties FD1-FD5 we can define the following rules:

FD1f: $\frac{Y \subseteq X}{X \rightarrow Y}$

FD2f: $\frac{X \rightarrow Y,\ Z \subseteq W}{XW \rightarrow YZ}$

FD3f: $\frac{X \rightarrow Y,\ Y \rightarrow Z}{X \rightarrow Z}$

FD4f: $\frac{X \rightarrow Y,\ YW \rightarrow Z}{XW \rightarrow Z}$

FD5f: $\frac{X \rightarrow Y,\ X \rightarrow Z}{X \rightarrow YZ}$

FD6f: $\frac{X \rightarrow YZ}{X \rightarrow Y}, \frac{X \rightarrow YZ}{X \rightarrow Z}$

Proposition

The rules FD4f, FD5f, FD6f can be expressed with FD1f, FD2f, FD3f.

We denote by $\mathcal{R}_1 = \{$FD1f, FD2f, FD3f$\}$,
and by $\mathcal{R}_2 = R_1 \cup \{$FD4f, FD5f, FD6f$\}$

Proposition

The rules FD4f, FD5f, FD6f can be expressed with the rules FD1f, FD2f, FD3f.

Some ideas of proof:

- For FD4f: We apply FD2f for $X \to Y$ and $W \subseteq W$. From that result and from $YW \to Z$, by using FD3f we obtain the result;
- FD5f: We apply FD2f for $X \to Y$ and $X \subseteq X$ and the same for $X \to Z$ and $Y \subseteq Y$. After that we use FD3f (transitivity) between the results;
- FD6f: from FD1f we have that $YZ \to Y$ and $YZ \to Z$ and from FD3f it results that $X \to Y$ and $X \to Z$

Armstrong's axioms

Armstrong defined (in *Dependency structures of database relationships* Proc. IFIP 74, Amsterdam, 580-583) the next inference rules (named *Armstrong's Axioms*):

A1: $\frac{}{A_1 \ldots A_n \rightarrow A_i}, i = \overline{1, n}$

A2: $\frac{A_1, \ldots A_m \rightarrow B_1, \ldots B_r}{A_1 \ldots A_m \rightarrow B_j}, j = \overline{1, r}$

$$\frac{A_1, \ldots A_m \rightarrow B_j,\ j = \overline{1,r}}{A_1 \ldots A_m \rightarrow B_1, \ldots B_r}$$

A3: $\frac{A_1, \ldots A_m \rightarrow B_1, \ldots B_r, \quad B_1, \ldots B_r \rightarrow C_1, \ldots C_p}{A_1 \ldots A_m \rightarrow C_1, \ldots C_p}$

where A_i, B_j, C_k are attibutes. We denote by $\mathcal{R}_A = \{$A1, A2, A3$\}$.

Obs: A3 is actually FD3f.

Proposition

The rules in $\mathcal{R}_1$ can be expressed with the ones in R_A and vice versa.

Notation:

$$\Sigma^+_{\mathcal{R}} = \{X \to Y | \Sigma \vdash_{\mathcal{R}} X \to Y\}$$

Proposition

Let $\mathcal{R}'_1$ and $\mathcal{R}'_2$ be two sets of rules such that $\mathcal{R}'_1$ can be expressed via $\mathcal{R}'_2$ and vice versa. Than $\Sigma^+_{\mathcal{R}'_1} = \Sigma^+_{\mathcal{R}'_2}$ for each set Σ of functional dependencies.

Therefore: $\Sigma^+_{\mathcal{R}_1} = \Sigma^+_{\mathcal{R}_A}$

Let $X \subseteq U$ and $\mathcal{R}$ a set of inference rules. We denote by

$$X^+_{\mathcal{R}} = \{A | \Sigma \vdash_{\mathcal{R}} X \to A\}$$

Lema

$\Sigma \vdash_{\mathcal{R}} X \to Y$ *if and only if* $Y \subseteq X^+_{\mathcal{R}_1}$.

Lema

Let Σ be a set of functional dependencies and $\sigma : X \to Y$ a functional dependency such that $\Sigma \nvdash_{\mathcal{R}_1} X \to Y$. Them there exists a relation r_σ that satisfy all functional dependencies from Σ and r_σ does not satisfy $X \to Y$.

Theorem

Fie Σ o multime de dependente functionale. Atunci exista o relatie r_0 ce satisface exact elementele lui $\Sigma^+_{\mathcal{R}_1}$, adica:

- *r_0 satisface τ, $\forall \tau \in \Sigma^+_{\mathcal{R}_1}$ si*
- *r_0 nu satisface γ, $\forall \gamma \notin \Sigma^+_{\mathcal{R}_1}$*

Multivalued dependencies

Example

Suppose that the person havin the CNP[1] $= 1$ (let's call him Adam) was admitted at two faculties and has a driving licence for both A and B cathegories:

	CNP	Faculty	Driving licence cathegory
r :	1	Computer Science	A
	1	Mathematics	B

What about the touple $(1, \text{Computer Science}, B)$? should this touple exist in the table ?

What other touple can be deduced from the existing ones ?

[1] Personal Numeric Code

Example

	CNP	Faculty	Driving licence cathegory
	1	Computer Science	A
r :	1	Mathematics	B
	1	Computer Science	B
	1	Mathematics	A

The touples t marked in red could be missing because they are redundant because they can be obtained from the first two touples.

Detecting Functional dependencies allowed us to remove one column because it could be reconstructed from the other columns.

Multivalued dependencies "help" us to remove lines (touples) that can be deduced from the other remaining lines.

Multivalued Dependencies - Defintion

Let $X, Y \subseteq U$. A multivalued dependency is denoted by $X \twoheadrightarrow Y$ (there is a double arrow there).

Definition

The relation r *over* $R[U]$ *satisfy the multivalued dependency* $X \twoheadrightarrow Y$ if for any two touples $t_1, t_2 \in r$ having $t_1[X] = t_2[X]$, there exists the touples t_3 and t_4 from r, such that:

- $t_3[X] = t_1[X], t_3[Y] = t_1[Y], t_3[Z] = t_2[Z]$;
- $t_4[X] = t_2[X], t_4[Y] = t_2[Y], t_4[Z] = t_1[Z]$

where $Z = U - XY$ (Z is also named $residue$).

Second example

r :

A	B	C	D			
a_1	b_1	c_1	d_1	t_1		t_1"
a_1	b_2	c_2	d_2	t_2		
a_1	b_1	c_1	d_2	t_3		t_2"
a_1	b_2	c_2	d_1	t_4		
a_2	b_3	c_1	d_1		t'_1, t'_4	
a_2	b_3	c_1	d_2		t'_2, t'_3	

r satisfy $A \twoheadrightarrow BC$

Because when $t_1[A] = t_2[A]$ we have that:
$t_3[A] = t_1[A], t_3[BC] = t_1[BC], t_3[D] = t_2[D]$ și
$t_4[A] = t_2[A], t_4[BC] = t_2[BC], t_4[D] = t_1[D]$

Question: How do we choose t_3", t_4" ?

Some equivalent definition

The relation r *over* $R[U]$ *satisfy the multivalued dependency* $X \twoheadrightarrow Y$, if for any $t_1, t_2 \in r$ with $t_1[X] = t_2[X]$ we have that $M_Y(t_1[XZ]) = M_Y(t_2[XZ])$

where $M_Y(t[XZ]) = \{t'[Y] | t' \in r, t'[XZ] = t[XZ]\}$ = values of Y from different touples in which the projections over XZ are equal (XZ is given as a parameter).

$$
\begin{array}{cccccc}
 & A & B & C & D & \\
\hline
 & a_1 & b_1 & c_1 & d_1 & = t_1 \\
 & a_1 & b_2 & c_2 & d_2 & = t_2 \\
r: & a_1 & b_1 & c_1 & d_2 & \\
 & a_1 & b_2 & c_2 & d_1 & \\
 & a_2 & b_3 & c_1 & d_1 & \\
 & a_2 & b_3 & c_1 & d_2 & \\
\hline
\end{array}
$$

$M_Y(t_1[AD]) = M_Y(t_2[AD]) = \{(b_1, c_1), (b_2, c_2)\}$

Facts

- If r satisfy the functional dependency $X \to Y$, then, for any $t \in r$, we have that $M_Y(t[XZ]) = \{t[Y]\}$.
- If r satisfy the functional dependency $X \to Y$, then r also satisfy the multivalued dependency $X \twoheadrightarrow Y$.
- If r satisfy the multivalued dependency $X \twoheadrightarrow Y$, then we can define a function $\psi : r[X] \to \mathcal{P}(r[Y])$, by $\psi(t[X]) = M_Y(t[XZ]), \forall t \in r$ (returns different values from the projection over Y).
  When r satisfy $X \to Y$, then $\psi : r[X] \to r[Y]$ (because the values projected over Y are the same in a functional dependency when the projections over X are equal).

Multivalued dependencies properties

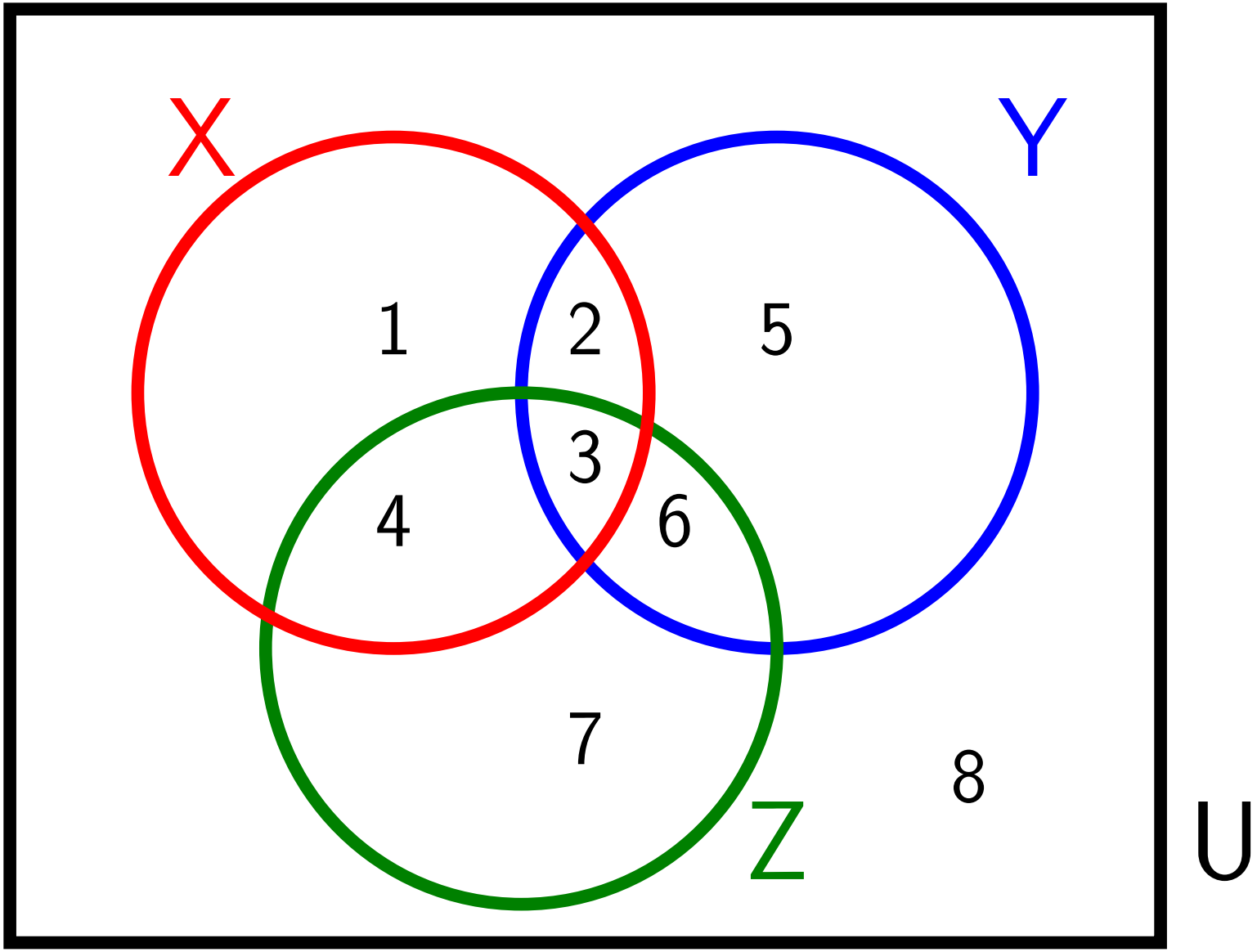
MVD0 (Complementariation) Let $X, Y, Z \subseteq U$, s.t. $XYZ = U$ and $Y \cap Z \subseteq X$. If r satisfy $X \twoheadrightarrow Y$, then r satisfy $X \twoheadrightarrow Z$.

MVD1 (Reflexivity) If $Y \subseteq X$, then any relation r satisfy $X \twoheadrightarrow Y$.

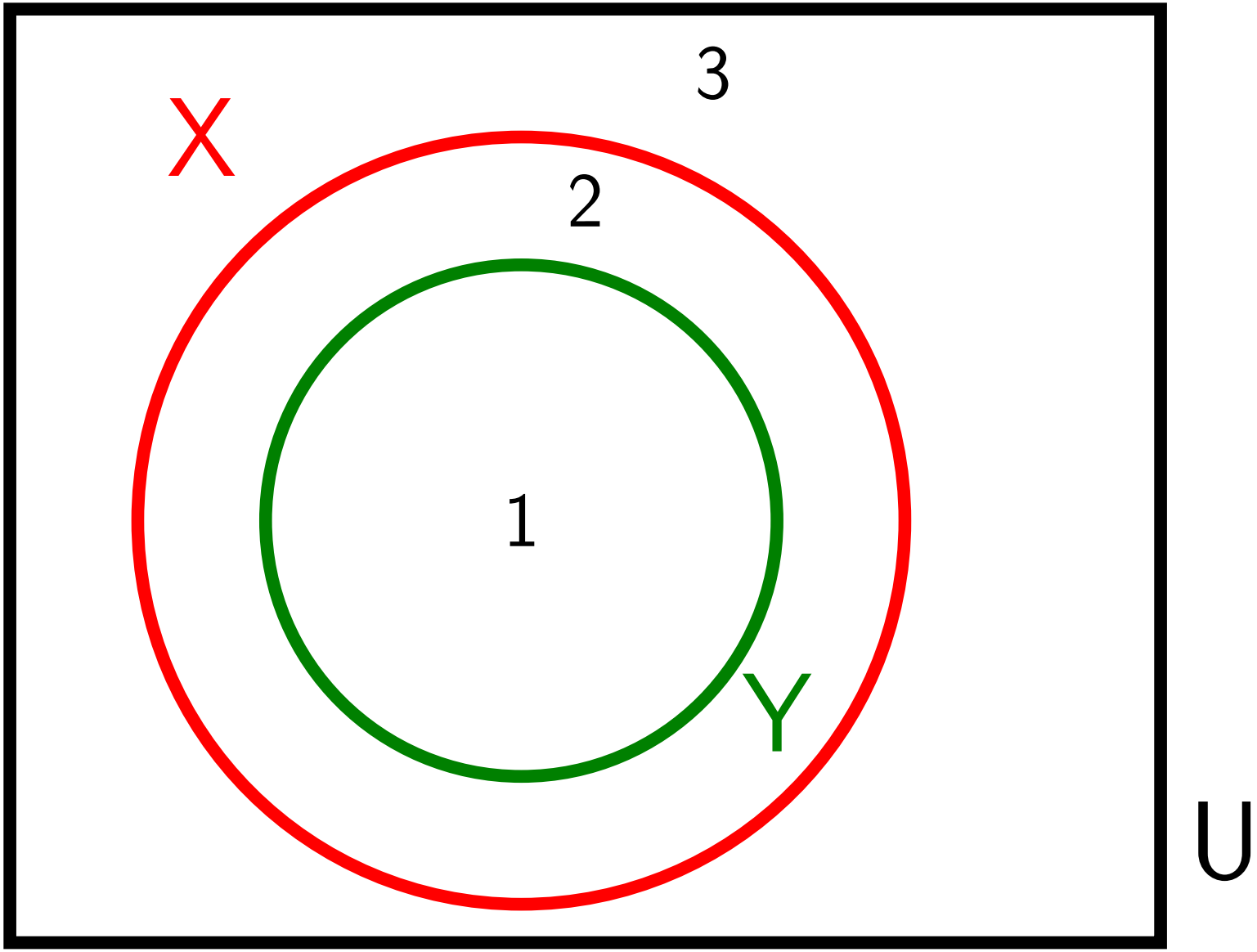
MVD2 (Extension) Let $Z \subseteq W$ and r satisfy $X \twoheadrightarrow Y$. Then r satisfy $XW \twoheadrightarrow YZ$

MVD3 (Transitivity) If r satisfy $X \twoheadrightarrow Y$ and $Y \twoheadrightarrow Z$, then r satisfy $X \twoheadrightarrow Z - Y$

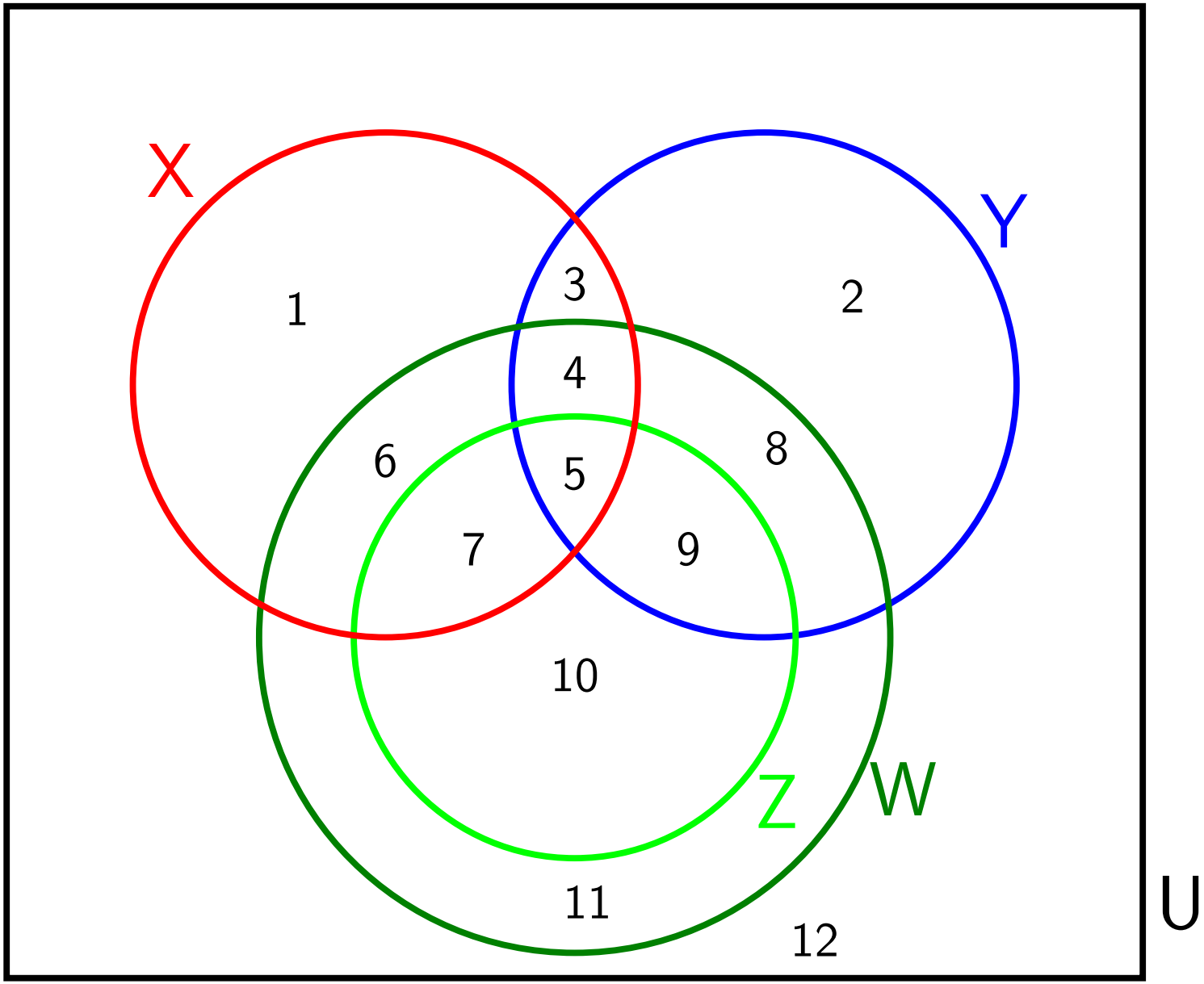
MVD0



MVD1



MVD2



Multivalued dependencies properties

MVD4 (Pseudotransitivity) If r satisfy $X \twoheadrightarrow Y$ and $YW \twoheadrightarrow Z$, then r satisfy and $XW \twoheadrightarrow Z - YW$.

MVD5 (Union) If r satisfy $X \twoheadrightarrow Y$ and $X \twoheadrightarrow Z$ then r satisfy $X \twoheadrightarrow YZ$.

MVD6 (Decomposition) If r satisfy $X \twoheadrightarrow Y$ and $X \twoheadrightarrow Z$, then r satisfy $X \twoheadrightarrow Y \cap Z$, $X \twoheadrightarrow Y - Z$, $X \twoheadrightarrow Z - Y$

Combined properties of multivalued dependencies

FD-MVD1. If r satisfy $X \to Y$, then r also satisfy $X \twoheadrightarrow Y$.

FD-MVD2. If r satisfy $X \twoheadrightarrow Z$ and $Y \to Z'$, with $Z' \subseteq Z$ and $Y \cap Z = \emptyset$, then r satisfy $X \to Z'$.

FD-MVD3. If r satisfy $X \twoheadrightarrow Y$ and $XY \twoheadrightarrow Z$, then r satisfy $X \to Z - Y$.

Inference rules

MVD0f: $\frac{XYZ=U,\ \ Y\cap Z\subseteq X,\ \ X\twoheadrightarrow Y}{X\twoheadrightarrow Z}$

MVD1f: $\frac{Y\subseteq X}{X\twoheadrightarrow Y}$

MVD2f: $\frac{Z\subseteq W,\ \ X\twoheadrightarrow Y}{XW\twoheadrightarrow YZ}$

MVD3f: $\frac{X\twoheadrightarrow Y,\ \ Y\twoheadrightarrow Z}{X\twoheadrightarrow Z-Y}$

MVD4f: $\frac{X\twoheadrightarrow Y, YW\twoheadrightarrow Z}{XW\twoheadrightarrow Z-YW}$

Inference rules

MVD5f: $\frac{X \twoheadrightarrow Y,\ X \twoheadrightarrow Z}{X \twoheadrightarrow YZ}$

MVD6f: $\frac{X \twoheadrightarrow Y,\ X \twoheadrightarrow Z}{X \twoheadrightarrow Y \cap Z,\ X \twoheadrightarrow Y - Z,\ X \twoheadrightarrow Z - Y}$

FD-MVD1f: $\frac{X \rightarrow Y}{X \twoheadrightarrow Y}$

FD-MVD2f: $\frac{X \twoheadrightarrow Z,\ Y \rightarrow Z',\ Z' \subseteq Z,\ Y \cap Z = \emptyset}{X \rightarrow Z'}$

FD-MVD3f: $\frac{X \twoheadrightarrow Y,\ XY \twoheadrightarrow Z}{X \twoheadrightarrow Z - Y}$

Proposition

Let $\mathcal{R}$ be a set of valid rules and γ a rule having the form $\frac{\alpha_1,\alpha_2,\ldots\alpha_k}{\beta}$, s.t. $\{\alpha_1,\ldots\alpha_k\} \vdash_{\mathcal{R}} \beta$, then the rule γ is also valid.

Proposition

Let $\mathcal{R}_{FM} = \{FD1f \ - \ FD3f$[2]$, \quad MVD0f \ - \ MVD3f, FD - MVD1f \ - \ FD - MVD3f\}$. We have that:

- *$FD - MVD3f$ can be expresses with the rest of the rule from $\mathcal{R}_{FM}$ and FD*
- *$MVD2f$ can be expressed with the other rules from $\mathcal{R}_{FM}$.*

Proposition

Regulile $MVD4f - MVD6f$ can be obtained from $MVD0f - MVD3f$

[2] the ones from functional dependencies

Theorem

Let Σ be a set of functional or multivalued dependencies and X a subset of attributes. Then there exists a partition of $U - X$ denoted by $Y_1 \dots Y_k$, s.t. for $Z \subseteq U - X$ we have that $\Sigma \vdash_{\mathcal{R}_{FM}} X \twoheadrightarrow Z$ iff Z is a union of some sets from the partition $\{Y_1, \dots Y_k\}$

Definition

For Σ a set of functional or multivalued dependencies and X a subset ofattributes, we define *the base of dependences for X w.r.t. Σ* the partition $B(\Sigma, X) = \{\{A_1\} \dots \{A_h\}, Y_1 \dots Y_k\}$, where $X = A_1, \dots A_h$, and $Y_1, \dots Y_k$ is the partition built in the previous theorem.

Observations

- We have that $\Sigma \vdash_{\mathcal{R}_{FM}} X \twoheadrightarrow Z$ iff Z is a union of elements from partition $B(\Sigma, X)$.
- Let $X^*_\Sigma = \{A | \Sigma \vdash_{\mathcal{R}_{FM}} X \rightarrow A\}$. Then, for any $A \in X^*_\Sigma$ we have that $\{A\} \in B(\Sigma, X)$.